

Static Code Analysis

1. Overview

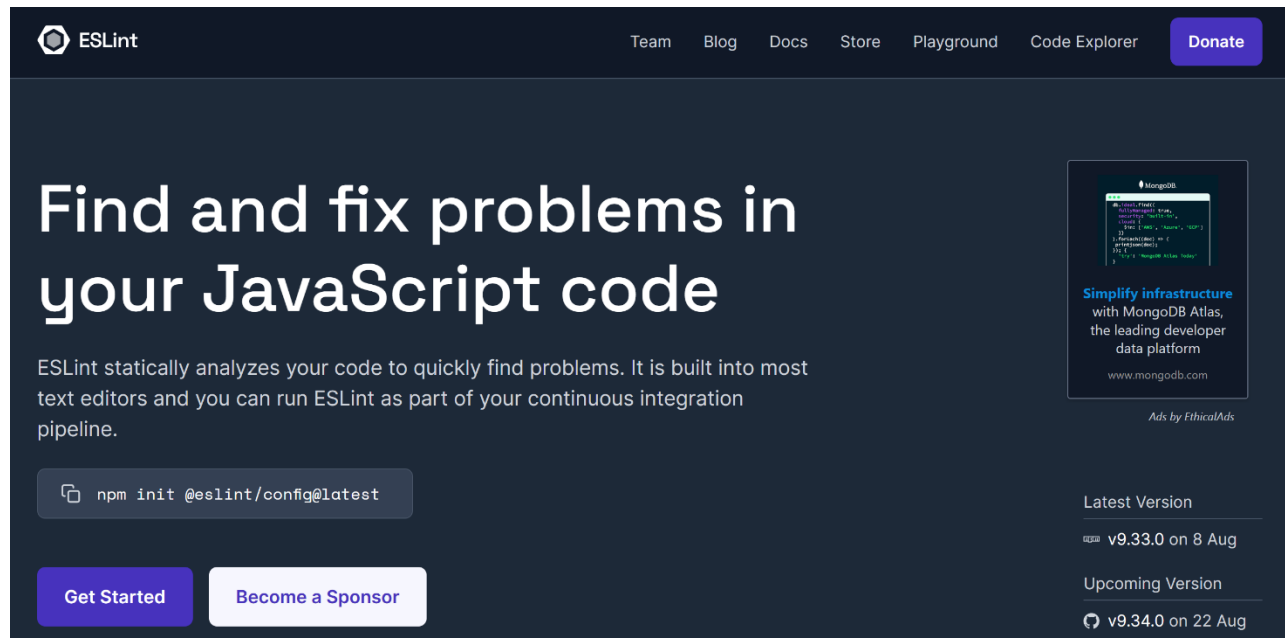


Figure 1: ESLint Website (<https://eslint.org/>)

In this lab, you are going to learn how to use ESLint and its security plugin to generate static code analysis report.

For ease of integration to Github Actions, you need a tool that generates reports in a format compatible with GitHub Code Scanning, the SARIF format.

2. Outcomes

Upon completion of this session, you should be able to

- Use ESLint to analysis source code
- Use eslint-plugin-security for security analysis
- Select a suitable Static Application Security Testing (SAST) tool for your team project
- Start incorporating Static Code Analysis into your team project

3. Prerequisites

1. Experience with web app development using node and npm.
2. Basic knowledge of web app testing using jest or similar frameworks.
3. Node and npm environment set up.

4. Configure the GitHub Actions Workflow

First install and configure eslint for your project. ESLint is used for javascript web application or node application.

There are two main configuration styles:

1. `eslinttrc.*`
 - a. Legacy config (JSON, YAML, or JS). Uses extends, plugins, and rules.
2. `eslint.config.mjs`
 - a. Modern Flat Config. Uses an array of config objects with direct plugin/rule imports.

The lab will focus on `eslint.config.mjs` configuration.

In addition, Github Actions uses SARIF report format (<https://github.com/github/codeql->

[action](#)).

The following instructions is a starting point for you to get it working for your project.

Please try out and adapt Eslint configuration, according to your requirements.

Install sample reactjs app

First clone or fork a sample reactjs app.

For example, you can try this app: <https://github.com/vipulpathak113/react-unit-testing#>

1. Please note that there may be errors with public repo project.
2. In the example app, there is a case error with import statement in the above public repo.

a. `import * as services from "../utils/Services";`

Install eslint and sarif formatter

```
npm install eslint@latest --save-dev  
npm install @microsoft/eslint-formatter-sarif --save-dev
```

You may need to fix security audit errors and deprecations errors depending on your setup.

Initialize eslint for your project

```
npx eslint --init
```

Edit and modify the `eslint.config.mjs` file. This is an iterative process when you set up the

test cases initially.

```
//import js from "@eslint/js";
//import globals from "globals";
import { defineConfig } from "eslint/config";
import reactPlugin from 'eslint-plugin-react';
import jestPlugin from 'eslint-plugin-jest';
import testingLibraryPlugin from 'eslint-plugin-testing-library';
import babelParser from '@babel/eslint-parser';

export default defineConfig([
  {
    files: ['**/*.js,jsx'],
    languageOptions: {
      parser: babelParser,
      parserOptions: {
        requireConfigFile: false,
        babelOptions: {
          presets: ['@babel/preset-react'],
        },
      },
    },
    plugins: {
      react: reactPlugin,
    },
    rules: {
      ...reactPlugin.configs.recommended.rules,
      'react/react-in-jsx-scope': 'off',
    },
    settings: {
      react: {
        version: 'detect', // Automatically picks up from package.json
      },
    },
  },
  {
    files: ['**/*.test.jsx'],
    plugins: {
      jest: jestPlugin,
      'testing-library': testingLibraryPlugin,
    },
    rules: {
      ...jestPlugin.configs.recommended.rules,
      ...testingLibraryPlugin.configs.react.rules,
      'testing-library/await-async-events': 'off',
    },
  },
]);
```

```
    },  
    languageOptions: {  
      globals: {  
        test: true,  
        expect: true,  
        describe: true,  
        beforeEach: true,  
        afterEach: true,  
        it: true,  
        jest: true,  
      },  
    },  
  },  
},  
]);
```

Install other required testing packages, e.g.

```
npm install eslint-config-react-app --save-dev
```

```
npm install eslint-plugin-testing-library --save-dev
```

```
npm install @babel/eslint-parser --save-dev
```

Test Locally

```
npx eslint .  
npx eslint . --format=@microsoft/eslint-formatter-sarif --output-  
file=reports/eslint-results.sarif
```

The public repo you used, would likely contain errors / warnings. Please attempt to fix any errors that is reported by eslint.

Remember to add eslint and test files to your repo

This step would depend on how you program your test case, eslint configurations and / or other test configurations (e.g. Jest).

E.g.

```
git add eslint.config.mjs
```

Github Actions Workflow

Copy the provided cda-eslint.yml to your github workflow directory.

This is a *template* configuration file, and you would need to update it or create a new configuration for your project.

Push this workflow file to `.github/workflows/eslint.yml`

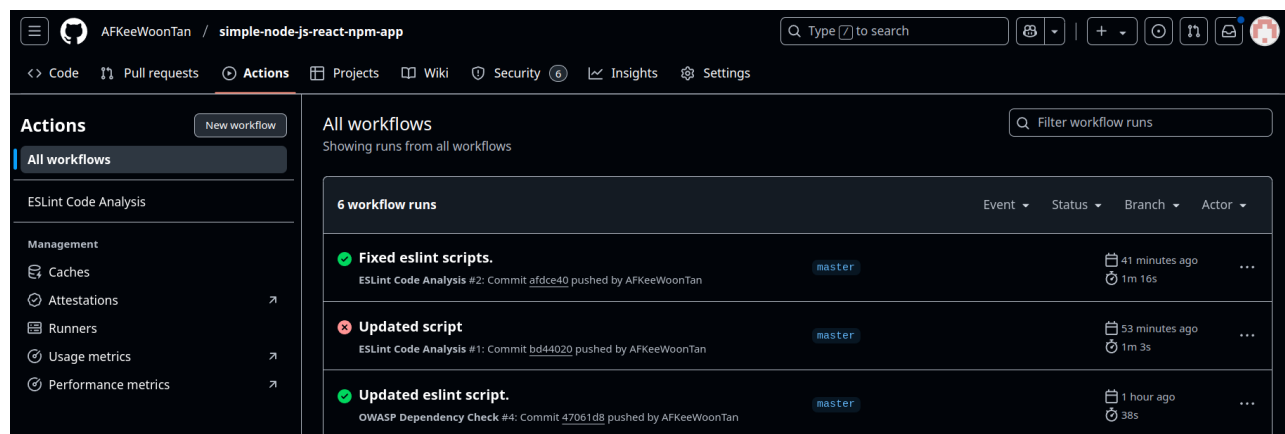
This workflow will:

- Run on push and pull requests to the `main` branch
- Install dependencies
- Run ESLint and fail if any issues are found

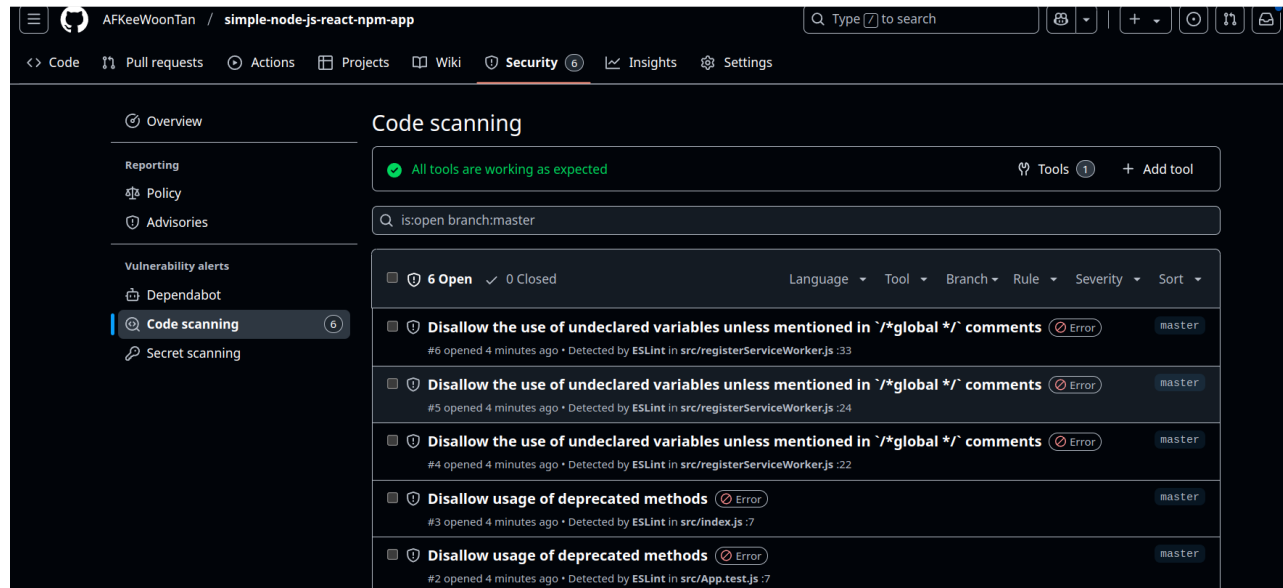
Trigger Github Actions Workflow

Commit and push your changes to Github

The status of your Github Workflow run can be viewed under Actions.



The ESLint report is found under Security.



5. ESLint Security Plugins

ESLint performs static code analysis, but its primary focus is on code quality, style, and potential bugs, rather than security vulnerabilities. However, it can be extended to detect security issues by using plugins like: `eslint-plugin-security`.

Detects common security issues in JavaScript, such as:

- Use of `eval()`
- Potential Regular Expression Denial of Service (ReDoS)
- Insecure use of `Buffer()`

NOTE: Using plugins with ESLint requires configuration according to your project requirements. Please explore the plugin documentations and define the necessary security rules needed. And treat the plugin instructions as templates that you should

adapt for your project.

5.1 ESLint-Plugin-Security

Installation of ESLint-Plugin-Security

```
npm install eslint-plugin-security --save-dev
```

Update eslint.config.mjs with security configuration

Import statement:

```
import pluginSecurity from "eslint-plugin-security";
```

Config:

```
plugins: {
  react: reactPlugin,
  security: pluginSecurity
},
rules: {
  ...reactPlugin.configs.recommended.rules,
  'react/react-in-jsx-scope': 'off',
  "security/detect-eval-with-expression": "error",
},
```

This config checks for eval statements.

Testing

If you are unable to get ESLint-Plugin-Security working for your project, you can test your config with a simple test.js class first.

```
// test.js
const expression = '1 + 1';
eval(`console.log(${expression})`); // This evaluates the expression 1 + 1
```

And test it locally using:

```
npx eslint test.js
```

```
npx eslint --debug test.js
```



```
npx eslint test.js --format=@microsoft/eslint-formatter-sarif --output-file=reports/eslint-results.sarif
```

5.2 eslint-plugin-security-node plugin

The eslint-plugin-security-node plugin focused on **Node.js security** vulnerabilities.

- Detects issues like:
 - Insecure HTTP headers
 - Improper handling of user input

Please try out this plugin and adapt it for your project, especially if you are using NodeJS.

The full configuration for this plugin is not provided in this lab document.

Installation of eslint-plugin-security-node

```
npm install eslint-plugin-security-node --save-dev
```

Update your `eslint.config.mjs`:

Import:

```
import securityNode from 'eslint-plugin-security-node';
```

Config:

```
plugins: {  
  'security-node': securityNode,  
},  
rules: {  
  'security-node/detect-crlf': 'error',  
},
```

5.3 eslint-plugin-no-unsanitized plugin

Helps prevent Cross-Site Scripting (XSS) by flagging improper DOM manipulation.

Please try out this plugin and adapt it for your project.

The full configuration for this plugin is not provided in this lab document.

Installation of eslint-plugin-no-unsanitized

```
npm install eslint-plugin-no-unsanitized --save-dev
```

Update your `eslint.config.mjs`:

Import:

```
import eslintPluginNoUnsanitized from 'eslint-plugin-no-unsanitized';
```

Config:

```
plugins: {  
  'no-unsanitized': eslintPluginNoUnsanitized,  
},  
rules: {  
  ...eslintPluginNoUnsanitized.configs.recommended.rules,  
},
```

6. Github CodeQL

To scan your repository for vulnerabilities using CodeQL, update and use the GitHub

Actions workflow file (`codeql.yml`) provided to you. **This is *template* file, to be used only as a starting point for your project configuration.**

7. Lab Notes

1. While ESLint helps with security best practices, it doesn't replace security-focused static analysis tools like:
 1. Semgrep – Lightweight, customizable security scanning.
 2. SonarQube – Detects security vulnerabilities in JavaScript/TypeScript.
2. Some other common tools that supports SARIF format include:
 1. Flake8 for Python
 2. Checkstyle for Java
 3. PMD for Java and Apex
 4. SonarCloud for multi-language analysis
3. We will explore SonarQube in the next lab.

8. Reference

Github SARIF format: <https://docs.github.com/en/code-security/code-scanning/integrating-with-code-scanning/sarif-support-for-code-scanning>

ESLint: [ESLint](#)

ESLint Plugin Security: <https://www.npmjs.com/package/eslint-plugin-security>

ESLint Plugin Security Node: <https://www.npmjs.com/package/eslint-plugin-security-node>

ESLint Plugin No Unsanitized: <https://www.npmjs.com/package/eslint-plugin-no-unsanitized>

Github CodeQL: <https://github.com/github/codeql-action>

END OF DOCUMENT